Charts and Visualization in Python using Matplotlib

Introduction

Data visualization is the process of representing data in a graphical or visual format. Instead of looking at large tables of numbers, charts and graphs help us quickly understand patterns, trends, relationships, and outliers.

In Data Science, visualization is one of the most important steps of **Exploratory Data Analysis (EDA)**. It helps analysts and data scientists communicate insights effectively and make data-driven decisions.

Python provides several visualization libraries, but the most fundamental and widely used is **Matplotlib**.

Learning Objectives

By the end of this chapter, you will be able to:

- Understand the importance of data visualization.
- Install and import Matplotlib.
- Create different types of charts.
- Customize charts with titles, labels, legends, and grids.
- Work with multiple datasets in a single chart.
- Save charts as image files.
- Choose the appropriate chart for different data types.

Why Do We Need Data Visualization?

Consider the following sales data.

Month	Sales
January	120
February	145
March	180
April	170
May	210

Viewing this data in a table makes it difficult to identify trends.

A line chart immediately shows:

- Sales increased steadily.
- A slight drop occurred in April.
- The highest sales were recorded in May.

Visualization makes complex information easier to understand.

Applications of Data Visualization

Data visualization is widely used in:

- Business Intelligence
- Financial Analysis
- Healthcare
- Marketing
- Education
- Sports Analytics
- Weather Forecasting
- Scientific Research

- Artificial Intelligence
- Machine Learning

Introduction to Matplotlib

Matplotlib is one of the most popular Python libraries used for creating static, animated, and interactive visualizations.

It supports:

- Line Charts
- Bar Charts
- Pie Charts
- Scatter Plots
- Histograms
- Box Plots
- Area Charts
- Multiple Plot Types

Installing Matplotlib

pip install matplotlib

Importing Matplotlib

import matplotlib.pyplot as plt

First Line Chart

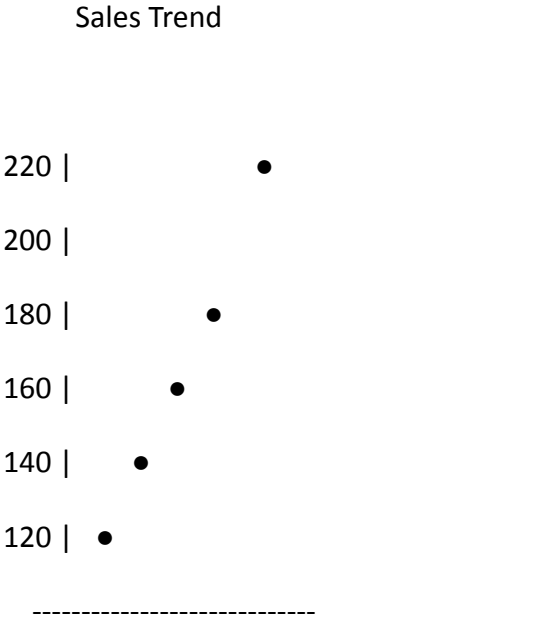
import matplotlib.pyplot as plt

```
months = ["Jan","Feb","Mar","Apr","May"]

sales = [120,145,180,170,210]


plt.plot(months, sales)


plt.show()
```

Components of a Chart

A chart generally contains:

- Title
- X-axis
- Y-axis
- Data
- Grid
- Legend



Adding a Title

plt.title("Monthly Sales Report")

Labeling Axes

plt.xlabel("Months")

plt.ylabel("Sales")

Adding Grid

plt.grid(True)

Changing Line Color

plt.plot(months, sales, color="red")

Common colors:

- red
- blue
- green
- orange
- purple
- black

Changing Line Style

plt.plot(months, sales, linestyle="--")

Styles:

- "-"
- "--"
- "-."
- ":"

Changing Marker

plt.plot(months, sales, marker="o")

Common markers:

- o
    - ○
- s
- ^
- x
    - ○

Complete Line Chart Example

```
import matplotlib.pyplot as plt


months = ["Jan","Feb","Mar","Apr","May"]

sales = [120,145,180,170,210]


plt.plot(

  months,

  sales,

  color="blue",

  marker="o",

  linestyle="-"
```

)

plt.title("Monthly Sales")

plt.xlabel("Months")

plt.ylabel("Sales")

plt.grid(True)

plt.show()

Bar Chart

A bar chart compares different categories.

Example

import matplotlib.pyplot as plt

students = ["Rahul","Priya","Aman","Neha"]

marks = [85,92,76,88]

plt.bar(students, marks)

plt.title("Student Marks")

plt.show()

Applications

- Sales comparison
- Student marks

- Product comparison
- Population comparison

Horizontal Bar Chart

plt.barh(students, marks)

Pie Chart

Pie charts show percentage distribution.

```
import matplotlib.pyplot as plt


subjects = ["Math","Science","English"]


marks = [40,35,25]


plt.pie(
  marks,
  labels=subjects,
  autopct="%1.1f%%"
)


plt.title("Subject Distribution")


plt.show()
```

Applications

- Market share
- Budget allocation
- Election results
- Expense analysis

Scatter Plot

Scatter plots show relationships between variables.

```
import matplotlib.pyplot as plt


hours = [1,2,3,4,5,6]

marks = [45,55,60,72,82,90]


plt.scatter(hours, marks)


plt.title("Study Hours vs Marks")


plt.xlabel("Study Hours")

plt.ylabel("Marks")


plt.show()
```

Applications

- Correlation analysis
- Machine Learning
- Pattern detection

Histogram

Histograms show data distribution.

```
import matplotlib.pyplot as plt


ages = [20,22,25,21,24,26,28,24,25,22,23,24]


plt.hist(ages)


plt.title("Age Distribution")


plt.show()
```

Applications

- Age distribution
- Salary distribution
- Exam scores

Box Plot

A box plot identifies spread and outliers.

```
import matplotlib.pyplot as plt


salary = [25000,28000,27000,26000,50000,30000,29000]


plt.boxplot(salary)
```

```
plt.title("Salary Distribution")


plt.show()
```

Applications

- Detecting outliers
- Statistical analysis
- Data cleaning

Area Chart

Area charts display cumulative trends.

```
import matplotlib.pyplot as plt


months = ["Jan","Feb","Mar","Apr","May"]

sales = [120,145,180,170,210]


plt.fill_between(months, sales)


plt.show()
```

Multiple Lines

```
import matplotlib.pyplot as plt


months = ["Jan","Feb","Mar","Apr","May"]
```

product1 = [100,120,140,160,180]

product2 = [90,130,135,170,200]

plt.plot(months, product1, label="Laptop")

plt.plot(months, product2, label="Mobile")

plt.legend()

plt.show()

Adding Legend

plt.legend()

Output

Laptop ——

Mobile ——

Saving Charts

plt.savefig("sales_chart.png")

The chart will be saved in the current working directory.

Figure Size

plt.figure(figsize=(8,5))

Width = 8 inches

Height = 5 inches

Common Chart Types

Chart	Purpose
Line Chart	Trends over time
Bar Chart	Compare categories
Pie Chart	Percentage distribution
Scatter Plot	Relationship between variables
Histogram	Data distribution
Box Plot	Outliers and spread
Area Chart	Cumulative values

Choosing the Right Chart

Situation	Recommended Chart
Monthly Sales	Line Chart
Product Comparison	Bar Chart
Market Share	Pie Chart
Height vs Weight	Scatter Plot
Salary Distribution	Histogram
Outlier Detection	Box Plot

Real-World Example

Suppose a company wants to analyze monthly revenue.

```
import matplotlib.pyplot as plt


months = ["Jan","Feb","Mar","Apr","May","Jun"]


revenue = [150000,170000,180000,210000,240000,260000]


plt.plot(months, revenue, marker="o")
```

plt.title("Company Revenue")

plt.xlabel("Months")

plt.ylabel("Revenue")

plt.grid(True)

plt.show()

This visualization quickly highlights growth trends and seasonal patterns.

Best Practices

- Add descriptive titles.
- Label both axes clearly.
- Use legends when plotting multiple datasets.
- Choose chart types appropriate for the data.
- Avoid excessive colors and decorations.
- Keep charts simple and readable.
- Use grids where they improve readability.

Common Mistakes

Forgetting `plt.show()`

plt.plot(x, y)

Without:

plt.show()

the chart may not display in some environments.

Wrong Chart Selection

Using a pie chart for time-series data is not appropriate.

Use a line chart instead.

Missing Labels

Charts without titles or axis labels are difficult to interpret.

Always include:

plt.title()

plt.xlabel()

plt.ylabel()

Summary

In this chapter, you learned:

- What data visualization is.
- Why visualization is important.
- Introduction to Matplotlib.
- Creating line, bar, pie, scatter, histogram, box, and area charts.
- Adding titles, labels, legends, and grids.
- Plotting multiple datasets.
- Saving charts.
- Choosing the appropriate chart type for different scenarios.

Key Takeaways

- **Data visualization** transforms raw data into meaningful visual insights.
- **Matplotlib** is the foundational visualization library in Python and is widely used in Data Science and Machine Learning.
- Different chart types serve different purposes: line charts for trends, bar charts for comparisons, pie charts for proportions, scatter plots for relationships, histograms for distributions, and box plots for detecting outliers.
- Well-designed charts should always include **titles, axis labels, legends (when needed), and appropriate formatting**.
- Mastering Matplotlib provides a strong foundation for advanced visualization libraries such as **Seaborn**, **Plotly**, and **Pandas plotting**, which build upon Matplotlib's capabilities.